

FRONTEND REFERENCE DOCUMENTATION

Enterprise Multi-Vendor E-Commerce Marketplace

A comprehensive reference covering architecture, component relationships, state management, routing, the API layer, the full checkout flow, and all 18 bugs resolved during development.

• **Generated:** June 11, 2026 • **Bugs Fixed:** 18 • **Sections:** 2 • **Scope:** Frontend Only

React 19

TypeScript

Vite

TanStack Router

TanStack Query

Zustand

Axios

Stripe.js

Tailwind CSS

React Hot Toast

Lucide React

Heroicons

SECTION 1



Bugs Fixed — 18 Total



Each bug card below documents the root cause, the fix applied, and all files changed. Bugs span the full stack (API configuration, TypeScript type errors, Stripe integration, React state, routing gaps, and UI regressions).

01

GET /api/v1/home returns **404 Not Found**

Bug

API / Backend

ROOT CAUSE

The home endpoint that the frontend expected — `GET /api/v1/home` — was never created in the backend. No route handler existed for that path, causing Express to fall through to the default 404 middleware on every page load.

FIX APPLIED

Created a new route file `backend/src/routes/home.ts` that handles `GET /api/v1/home`. The handler returns featured products, active categories, promotional banners, and site statistics. The route was then imported and registered in the main Express application.

FILES CHANGED`backend/src/routes/home.ts (created)``backend/src/app.ts`

02

GET /api/v1/cart returns **400 Bad Request** — missing **x-session-id** header

Bug

API / Backend

Auth / Session

ROOT CAUSE

Guest cart functionality on the backend requires an `x-session-id` header to identify anonymous sessions. The frontend Axios instance had no logic to generate or attach this header. Additionally, CORS was configured without `X-Session-ID` in its `allowedHeaders` list, so the header was being stripped by CORS preflight even when manually sent.

FIX APPLIED

In the Axios request interceptor inside `api.ts`, a UUID is now generated once with `crypto.randomUUID()`, stored in `localStorage` under the key `'session-id'`, and automatically attached as `x-session-id` on every outgoing request. On the backend, the CORS configuration was updated to explicitly allow `X-Session-ID` in `allowedHeaders`.

FILES CHANGED`frontend/src/utils/api.ts``backend/src/configs/app.ts`

03

Runtime crash: `product.averageRating.toFixed` is not a function

Bug

API / Backend

ROOT CAUSE

Prisma stores numeric columns like `averageRating` and `price` using its internal `Decimal` type. When Prisma serializes a model to JSON (e.g. in an API response), `Decimal` fields are converted to a **plain string** rather than a JavaScript number. The frontend assumed `averageRating` was a `number`, so calling `.toFixed(1)` on a string threw a runtime `TypeError`.

FIX APPLIED

Added a global override in the Prisma database config:

```
Decimal.prototype.toJSON = function() { return  
parseFloat(this.toString()); } This ensures every Decimal field is  
serialized as a native JavaScript number when JSON.stringify is called,  
so all callers on the frontend can use numeric methods like .toFixed()  
safely.
```

FILES CHANGED`backend/src/configs/database.ts`

04

TanStack Query error: "Query data cannot be undefined" — affects 17 files

Bug

API Layer

ROOT CAUSE

The Axios instance in `api.ts` uses a response interceptor that **already unwraps** the standard API envelope `{ success, data, message, meta }`, returning the inner `data` payload. All 17 affected files were still performing a second unwrap: `api.get<ApiResponse<X>>(...).then(r => r.data.data)`. The double-unwrap caused `r.data.data` to be `undefined` since `r.data` was already the final payload — TanStack Query rejects `undefined` query data by design.

FIX APPLIED

All 17 query functions were updated to the correct single-unwrap pattern: `api.get<X>(...).then(r => r.data)`. Generic type parameters were updated from `ApiResponse<X>` to `X` throughout, removing the redundant wrapper type.

FILES CHANGED

17 service / query files

features/products/hooks.ts

features/cart/hooks.ts

features/orders/hooks.ts

features/auth/hooks.ts

features/vendor/hooks.ts

features/admin/hooks.ts

+ 11 more

05

OrderSummary crashes — incorrect type wrapper around cart data

Bug

UI / Component

ROOT CAUSE

`CheckoutForm` was typed with a local `CartData` interface that had a nested structure: `{ summary: CartSummary }`. The template accessed data as `cart.summary.subtotal`, `cart.summary.total`, etc. However, the API returns a flat `CartSummary` object with no `summary` key, so all property accesses on `cart.summary` evaluated to `undefined`, causing downstream crashes in `OrderSummary`.

FIX APPLIED

Removed the intermediate `CartData` interface entirely. The `useQuery` return type was changed to `CartSummary` directly. All `cart.summary.X` references were replaced with `cart.X` to match the flat response shape.

FILES CHANGED`frontend/src/features/checkout/CheckoutForm.tsx``frontend/src/types/cart.types.ts`

06

Category filter returns empty results when clicking a parent category

Bug

API / Backend

ROOT CAUSE

The product repository filtered by an **exact** `categoryId` match. The category hierarchy has parent and child (subcategory) nodes, and products are only ever assigned to leaf subcategories, not parent categories. Selecting a parent category produced zero results because no product directly carried that parent's ID.

FIX APPLIED

Added a recursive helper function `getCategoryDescendantIds(categoryId)` to `products.repository.ts`. It walks the category tree downward and collects all descendant IDs. The Prisma query was updated from `categoryId: id` to `categoryId: { in: categoryIds }`, where `categoryIds` is the full descendant set including the selected parent.

FILES CHANGED`backend/src/repositories/products.repository.ts`

07

isFeatured Zod transform silently applies **false** filter to all requests

Bug

API / Backend

ROOT CAUSE

The Zod schema for product query parameters defined: `isFeatured: z.string().optional().transform(v => v === 'true')`. When `isFeatured` was absent from the request (i.e., `undefined`), the transform evaluated `undefined === 'true'` which is `false`. Zod passed this `false` value through, and the database query added a `WHERE isFeatured = false` clause on every request that omitted the parameter — silently filtering out all featured products from regular listings. The same class of bug existed in numeric query parameters such as `minPrice` and `maxPrice`.

FIX APPLIED

All boolean and numeric Zod transforms were updated to preserve `undefined` when the input is absent: `transform(v => v !== undefined ? v === 'true' : undefined)`. This ensures that an omitted filter parameter results in `undefined`, which Prisma ignores entirely rather than filtering on.

FILES CHANGED`backend/src/validators/product.validators.ts`

08

Order creation returns 400 — field name mismatch between frontend and backend

Bug

API / Backend

ROOT CAUSE

The backend `createOrderSchema` (Zod) expected an `addressId` field — a reference to a previously-saved address record in the database. The frontend's checkout flow sent an inline `shippingAddress` object containing street, city, state, postal code, and country directly in the request body. Zod validation failed because neither shape matched.

FIX APPLIED

The `createOrderSchema` was updated to accept an inline `shippingAddress` object (with a nested Zod shape matching the frontend payload). The order service and TypeScript interface types were updated to reflect the new shape. The frontend checkout `AddressForm` submission logic remained unchanged.

FILES CHANGED

`backend/src/validators/order.validators.ts``backend/src/services/order.service.ts``backend/src/types/order.types.ts`

09

Stripe Elements throws "invalid clientSecret" for mock payment intents

Bug

UI / Checkout

ROOT CAUSE

When Stripe is not configured in the environment, the backend returned a mock `clientSecret` formatted as `pi_mock_xxx_secret_demo`. Stripe.js strictly validates the `clientSecret` format before mounting `<Elements>`: it expects a string in the pattern `pi_[id]_secret_[key]` with exactly one underscore separating each segment. The extra underscores in `pi_mock_xxx` caused Stripe's internal regex to reject the secret before any network call was made.

FIX APPLIED

The backend mock secret format was changed to `mock_{orderId}` (no `pi_` prefix). In `CheckoutForm.tsx`, a guard was added: if `clientSecret.startsWith('mock_')` the component skips the Stripe `<Elements>` tree entirely and renders a **"Demo Mode — Complete Order"** button. Clicking this button calls `POST /payments/confirm` directly with a synthetic payment intent ID, bypassing Stripe altogether.

FILES CHANGED

`frontend/src/features/checkout/CheckoutForm.tsx``backend/src/services/payment.service.ts`

10

Cart not cleared after successful checkout

Bug

UI / State

ROOT CAUSE

The `confirmOrderMutation.onSuccess` callback only called `navigate('/order-success')`. It never cleared the Zustand cart store (which persists to `localStorage`) nor invalidated the `['cart']` TanStack Query cache. Users arriving at the order success page still saw a non-zero cart count in the header, and refreshing the page re-fetched stale cart data.

FIX APPLIED

Added two side-effects inside `onSuccess` before navigation: `clearLocalCart()` from the Zustand cart store (resets `items`, `itemCount`, `subtotal`, `total`) and `queryClient.invalidateQueries({ queryKey: ['cart'] })` to force a fresh fetch from the server on next render.

FILES CHANGED

`frontend/src/features/checkout/CheckoutForm.tsx`

11

Profile dropdown menu items do nothing when clicked

Bug

UI / Navigation

ROOT CAUSE

Every `onClick` handler on the profile dropdown menu items (Profile, My Orders, Vendor Dashboard, Admin Panel, Sign Out) was set to the no-op arrow function `() => {}`. This was leftover placeholder code from the initial scaffold — no navigation or action was ever wired up.

FIX APPLIED

Imported `useNavigate` from TanStack Router in the `Header` component. Each menu item's `onClick` was wired to the correct route: `/profile`, `/orders`, `/vendor/dashboard`, `/admin`. The Sign Out item calls `clearAuth()` from the auth store then navigates to `/`. Role-based items (Vendor Dashboard, Admin Panel) are conditionally rendered using `isVendor()` and `isAdmin()`.

FILES CHANGED

`frontend/src/components/shared/Header.tsx`

12

Pay button shows incorrect amount (100× too large)

Bug

UI / Checkout

ROOT CAUSE

`PaymentForm` displayed the charge amount with the expression `(amount / 100).toFixed(2)`, treating the API-returned dollar value as if it were already in Stripe's smallest-unit (cents) format. The API returns `amount` in dollars (e.g., `49.99`), so dividing by 100 produced `$0.50` instead of `$49.99`.

FIX APPLIED

Changed the display expression to `amount.toFixed(2)` — no division. The button now correctly shows the dollar amount returned from the order creation endpoint.

FILES CHANGED

`frontend/src/features/checkout/PaymentForm.tsx`

13

Footer links cause page scroll-to-top instead of navigation

Bug

UI / Navigation

ROOT CAUSE

All footer links were rendered as `<a href="#">`. The `#` hash navigates to the top of the current page (browser default for empty anchors), producing an unwanted scroll jump. None of the links were connected to TanStack Router, so clicking any footer link was non-functional.

FIX APPLIED

Links with registered routes (Products, Categories, About, Contact, Terms, Privacy) were replaced with TanStack Router `<Link>` components pointing to the correct `to` path. Links for pages not yet implemented were replaced with `<span className="cursor-pointer">` elements so they are inert — no scroll jump, no dead navigation.

FILES CHANGED

`frontend/src/components/shared/Footer.tsx`

14

Page content renders flush against screen edges — no container or padding

Bug

UI / Layout

ROOT CAUSE

The `<main>` element inside `MainLayout` rendered route content as a full-width block with no horizontal padding or max-width constraint. On wide viewports, content stretched edge-to-edge. On narrow viewports, text was flush against the viewport boundary.

FIX APPLIED

Wrapped the `<Outlet />` inside `<div className="container-narrow py-8">` in `MainLayout`. The `container-narrow` utility class applies a `max-width` and centered horizontal margin; `py-8` adds vertical breathing room above and below the page content.

FILES CHANGED

`frontend/src/layouts/MainLayout.tsx`

15

Vendor and admin sidebar links return 404 — routes not registered

Bug

UI / Routing

Vendor

Admin

ROOT CAUSE

The TanStack Router route tree only defined `/vendor/dashboard` and `/admin` as routes. The sidebar navigation components linked to 12 additional paths — `/vendor/products`, `/vendor/orders`, `/vendor/analytics`, `/admin/users`, `/admin/vendors`, `/admin/products`, `/admin/orders`, and others — none of which had corresponding route definitions. TanStack Router rendered its 404 component for every one of them.

FIX APPLIED

All 12 missing routes were registered in `routes/index.tsx`, each wrapped in the appropriate role guard (`VendorSubRoute` or `AdminRoute`), and mapped to their respective page components in `pages/`. Stubs were created for pages that did not yet exist.

FILES CHANGED`frontend/src/routes/index.tsx``frontend/src/pages/vendor/*``frontend/src/pages/admin/*`

16

Customer users see the vendor sidebar instead of the "Become a Vendor" page

Bug

UI / Access Control

Customer

Vendor

ROOT CAUSE

`VendorDashboardPage` was unconditionally wrapped in `DashboardLayout`, which renders the sidebar navigation with vendor-specific links. When a customer navigated to `/vendor/dashboard`, they saw the full dashboard sidebar UI even though they are not vendors and have no vendor data to display. The page should instead explain how to become a vendor.

FIX APPLIED

Introduced a `VendorGate` wrapper component. It reads the current user's role from `useAuth()`: if the role is `VENDOR` or `ADMIN` it renders the children inside `DashboardLayout`; otherwise it renders a plain "Become a Vendor" landing page with an apply CTA — no sidebar, no vendor UI. `VendorDashboardPage` was wrapped with `VendorGate` in the route definition.

FILES CHANGED`frontend/src/features/vendor/VendorDashboard.tsx``frontend/src/routes/index.tsx`

17

Vendor "Apply Now" button submits empty hardcoded data

Bug

UI / Form

Vendor

ROOT CAUSE

The vendor application mutation was wired to fire immediately on button click with hardcoded empty values: `applyForVendor({ businessName: '', businessEmail: '' })`. There was no form collecting actual input from the user. Every "Apply Now" click sent an invalid empty payload to the API, which rejected it with a validation error.

FIX APPLIED

Replaced the single button with a complete application form using controlled React state (`useState`). The form collects: business name, business email, phone number, and a description/about field. On submit the form values are passed to the mutation, replacing the hardcoded empty object. Basic client-side validation was also added.

FILES CHANGED

`frontend/src/features/vendor/VendorDashboard.tsx`

18

"Already have a vendor application" API error not handled — stale JWT causes re-submission loop

Bug

UI / UX

Auth / State

Vendor

ROOT CAUSE

After a successful vendor application submission the backend promotes the user's role to `VENDOR` in the database, but the frontend's JWT is not updated in real time. The Zustand auth store still holds the old token claiming role `CUSTOMER`. Subsequent visits to `/vendor/dashboard` showed the application form again (because role is still `CUSTOMER` in the store), and re-submitting produced a confusing backend error: "you already have a pending vendor application." There was also no `onError` handler to intercept or display this message gracefully.

FIX APPLIED

Two changes were made together: (1) **onSuccess:** After a successful application, call `GET /users/me` immediately and dispatch `updateUser()` with the fresh user object from the server. This updates the role in the Zustand auth store and in `localStorage`, so the UI transitions to the vendor view without requiring a page refresh or re-login. (2) **onError:** If the API error message contains "already have a vendor application", render an inline `<PendingReview />` component instead of showing a generic error toast. `PendingReview` displays a friendly status message explaining that the application is under review.

FILES CHANGED

`frontend/src/features/vendor/VendorDashboard.tsx``frontend/src/store/auth.store.ts`

SECTION 2



Frontend Architecture

Project Structure

The `frontend/src/` directory follows a features-first layout. Each business domain (auth, cart, checkout, orders, products, vendor, admin) owns its components, hooks, and related logic in `features/`. Cross-cutting concerns (routing, layouts, shared UI primitives, global stores, and the API layer) live in top-level sibling directories.

```

frontend/src/ |— components/ | |— shared/ # Header, Footer, Sidebar,
ProtectedRoute, RoleGuard | |— ui/ # Button, Avatar, Dropdown, Skeleton, Spinner,
Pagination ... | |— features/ | |— auth/ # LoginForm, RegisterForm, hooks
(useLogin, useRegister) | |— cart/ # CartDrawer, CartItem, useCart hook | |—
checkout/ # CheckoutForm, AddressForm, PaymentForm, OrderSummary | |— orders/ #
OrderList, OrderStatus badge, useOrders hook | |— products/ # ProductCard,
ProductSearch, RelatedProducts, ProductReviews | |— vendor/ # VendorDashboard,
VendorProducts, VendorOrders, VendorAnalytics, ProductForm | |— admin/ #
AdminDashboard, UserManagement, VendorApproval, ProductModeration | |— hooks/ #
useAuth, usePagination, useDebounce |— layouts/ # MainLayout, AuthLayout,
DashboardLayout |— pages/ # One file per route: Home, Products, ProductDetail,
Cart, Checkout ... |— routes/ | |— index.tsx # Complete route tree – TanStack
Router code-based |— services/ # API service modules per domain (order.service,
vendor.service ...) |— store/ | |— auth.store.ts # Zustand + persist – user,
tokens, isAuthenticated | |— cart.store.ts # Zustand + persist – items, itemCount,
subtotal, isOpen |— types/ # TypeScript interfaces per domain |— utils/ |—
api.ts # Axios instance – interceptors, envelope unwrap |— format.ts # Currency,
date, number formatters |— cn.ts # Tailwind className merge utility |— storage.ts
# localStorage helpers

```



Convention: Every feature directory is self-contained. A feature should not import from another feature's internals. Cross-feature communication happens through shared stores (`store/`), shared hooks (`hooks/`), or shared UI primitives (`components/ui/`).

Routing Architecture

Routing is handled entirely by **TanStack Router** using a *code-based* route tree defined in `routes/index.tsx`. There are no file-based conventions. Three distinct layout branches exist; each branch applies different chrome (header, sidebar, auth card) and different access guards.



mainLayoutRoute

Wraps all public-facing and customer pages. Renders a sticky header, a centred container, and a footer. Used for: Home, Products, Product Detail, Cart, Checkout, Orders, Profile, static pages.

 authLayoutRoute

Centered card layout — no header or footer. Used exclusively for: Login, Register, Forgot Password, Reset Password.

 Vendor / Admin Sub-routes

Vendor and admin pages do not use a separate top-level layout route. Instead, the route component itself is wrapped in a role-aware gate:

- **VendorGate** — Reads the user's role. Renders `DashboardLayout` (sidebar) for VENDOR/ADMIN; renders a plain "Become a Vendor" page for CUSTOMER.
- **VendorSubRoute** — Strict guard; redirects non-vendors to `/vendor/dashboard` (which then lands on the gate).
- **AdminRoute** — Strict guard; redirects non-admins to `/`.

Route guard wrappers — all three are React components wrapping `<Outlet />`:

GUARD COMPONENT	CONDITION	REDIRECT ON FAIL
<code>ProtectedRoute</code>	<code>isAuthenticated === true</code>	<code>/login?redirect=...</code>
<code>RoleGuard</code>	User role matches required role prop	<code>/</code> (or configurable)
<code>VendorGate</code>	Role is VENDOR or ADMIN → <code>DashboardLayout</code> ; CUSTOMER → Apply page	N/A (renders alternative UI)
<code>VendorSubRoute</code>	Role is VENDOR or ADMIN	<code>/vendor/dashboard</code>
<code>AdminRoute</code>	Role is ADMIN	<code>/</code>

Registered route tree (selected):

/ `HomePage`

public

<code>/products</code>	<code>ProductsPage</code>	public
<code>/products/:slug</code>	<code>ProductDetailPage</code>	public
<code>/cart</code>	<code>CartPage</code>	public
<code>/checkout</code>	<code>CheckoutPage → ProtectedRoute</code>	auth required
<code>/orders</code>	<code>OrdersPage → ProtectedRoute</code>	auth required
<code>/orders/:id</code>	<code>OrderDetailPage → ProtectedRoute</code>	auth required
<code>/profile</code>	<code>ProfilePage → ProtectedRoute</code>	auth required
<code>/order-success</code>	<code>OrderSuccessPage → ProtectedRoute</code>	auth required
<code>/login</code>	<code>LoginPage (AuthLayout)</code>	public
<code>/register</code>	<code>RegisterPage (AuthLayout)</code>	public
<code>/vendor/dashboard</code>	<code>VendorDashboardPage → VendorGate</code>	auth required
<code>/vendor/products</code>	<code>VendorProductsPage → VendorSubRoute</code>	vendor
<code>/vendor/orders</code>	<code>VendorOrdersPage → VendorSubRoute</code>	vendor
<code>/vendor/analytics</code>	<code>VendorAnalyticsPage → VendorSubRoute</code>	vendor
<code>/admin</code>	<code>AdminDashboardPage → AdminRoute</code>	admin
<code>/admin/users</code>	<code>UserManagementPage → AdminRoute</code>	admin
<code>/admin/vendors</code>	<code>VendorApprovalPage → AdminRoute</code>	admin
<code>/admin/products</code>	<code>ProductModerationPage → AdminRoute</code>	admin
<code>/admin/orders</code>	<code>AdminOrdersPage → AdminRoute</code>	admin

State Management

Global client state is managed by two **Zustand** stores, both configured with the `persist` middleware to survive page refreshes. Server state (products, orders, cart data from the API) is managed by **TanStack Query**.

**auth.store — persisted to localStorage key****'auth-store'****user****User | null**

Full user profile object (id, email, name, role, avatar)

accessToken**string | null**

JWT access token injected into every API request via Authorization header

refreshToken**string | null**

JWT refresh token used to obtain a new access token on 401

isAuthenticated**boolean**

Derived from whether accessToken and user are non-null

setAuth()**action**

Sets user + both tokens on login / token refresh

updateUser()**action**

Partially updates the user object (used after /users/me refresh — Bug 18)

clearAuth()**action**

Resets all fields to null/false on logout or 401

setAccessToken()**action**

Updates only the access token after a silent refresh

**cart.store — persisted to localStorage (isOpen excluded)****items****CartItem[]**

Array of cart line items (productId, quantity, price, name, image)

itemCount**number**

Total quantity of all items — displayed in the header badge

subtotal

number

Sum of item prices before discounts/shipping

total**number**

Final amount including tax, shipping, coupon deductions

coupon**Coupon | null**

Applied coupon code and discount amount if present

isOpen**boolean**

Controls CartDrawer slide-over visibility — NOT persisted

setCart()**action**

Syncs full cart data from the server response into the store

clearLocalCart()**action**

Resets items/count/totals to zero — called on checkout success (Bug 10)

toggleCart()**action**

Flips isOpen — called from cart icon in Header



Pattern: Zustand stores hold *UI state* and *auth credentials*. Actual server data (product lists, order history, cart contents from the API) is always fetched and cached by TanStack Query. The cart store is synced via `setCart()` inside the cart query's `onSuccess` callback, making the store a projection of server state rather than the source of truth.

**API Layer — `utils/api.ts`**

A single **Axios instance** is the only mechanism for all HTTP calls. It handles authentication, session tracking, response normalisation, and automatic logout on 401. No raw `fetch()` calls exist in the codebase.

```
// Axios instance – simplified illustration
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  headers: { 'Content-Type': 'application/json' },
});
```

TYPESCRIPT

```
// — Request interceptor —————
api.interceptors.request.use((config) => {
  // 1. Auth token
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;

  // 2. Guest session tracking (Bug #2 fix)
  let sessionId = localStorage.getItem('session-id');
  if (!sessionId) {
    sessionId = crypto.randomUUID();
    localStorage.setItem('session-id', sessionId);
  }
  config.headers['x-session-id'] = sessionId;
  return config;
});

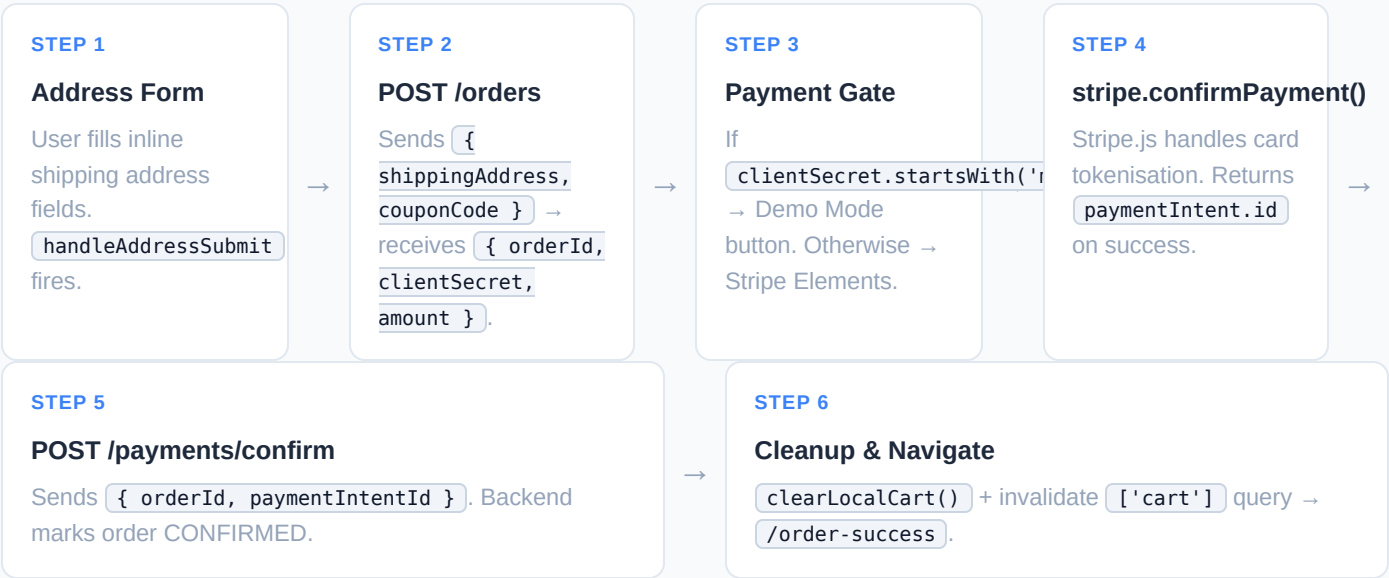
// — Response interceptor —————
api.interceptors.response.use(
  (response) => {
    // Unwrap API envelope: { success, data, message, meta }
    // Returns the inner payload – callers use api.get<T>().then(r => r.data)
    return response.data;
  },
  (error) => {
    if (error.response?.status === 401) {
      // Trigger global logout – ProtectedRoute listens for this event
      window.dispatchEvent(new Event('auth:logout'));
    }
    return Promise.reject(error);
  }
);
```

CALLER PATTERN	RETURN TYPE	NOTES
<code>api.get<User>('/users/me')</code>	<code>Promise<{ data: User, meta?, success, message }></code>	Use <code>.then(r => r.data)</code> to get the User
<code>api.get<Product[]>('/products')</code>	<code>Promise<{ data: Product[], meta: PaginationMeta }></code>	<code>meta</code> carries pagination info

CALLER PATTERN	RETURN TYPE	NOTES
<code>api.post<Order>('/orders', body)</code>	<code>Promise<{ data: Order }></code>	Mutations follow the same pattern

Checkout Flow

The checkout experience is a multi-step process orchestrated entirely within `CheckoutForm.tsx`. It progresses from address collection to payment and finally order confirmation, with a special Demo Mode path when Stripe is not configured.



```
// CheckoutForm - abridged confirm flow
const confirmMutation = useMutation({
  mutationFn: ({ orderId, paymentIntentId }) =>
    confirmOrder({ orderId, paymentIntentId }),

  onSuccess: () => {
    clearLocalCart();
    queryClient.invalidateQueries({ queryKey: ['cart'] }); // Bug #10 fix
    navigate({ to: '/order-success' });
  },
});

// Demo Mode path (Bug #9 fix)
if (clientSecret.startsWith('mock_')) {
  return (
    <DemoModeButton
```

```
onClick={() => confirmMutation.mutate({
  orderId,
  paymentIntentId: `demo_${Date.now()}`,
})}
/>
);
}
```

Key Component Relationships

The following diagrams and descriptions trace the data flows for the most architecturally significant component clusters.

Header Component

`Header` is the single most connected component in the application. It consumes both global stores and exposes role-aware navigation.

DEPENDENCY	WHAT IT PROVIDES	USED FOR
<code>useAuth()</code>	<code>user</code> , <code>isAuthenticated</code> , <code>clearAuth</code> , <code>isAdmin()</code> , <code>isVendor()</code>	Show/hide login button, build role-based dropdown menu items, handle Sign Out
<code>useCartStore()</code>	<code>itemCount</code> , <code>toggleCart</code>	Cart icon badge counter; open/close CartDrawer on click
<code>useNavigate()</code>	TanStack Router navigate function	Profile dropdown item navigation (Bug #11 fix)
<code><Dropdown></code>	UI primitive from <code>components/ui/</code>	User avatar menu with profile, orders, role-specific pages

ProductSearch / Products Page — URL-driven Filter State

Filter state (search query, category, price range, sort) lives exclusively in the URL as search params, not in local component state. This makes every filtered view shareable via link and navigable via the

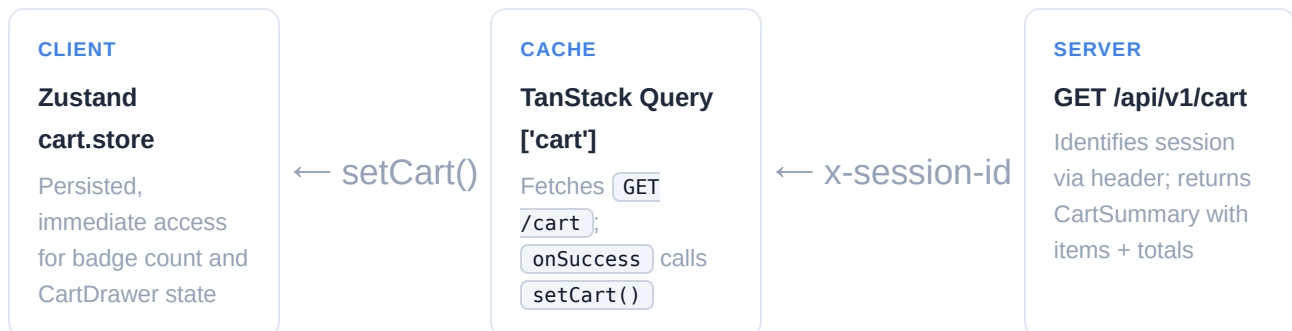
browser back button.

- **Reading state:** `useSearch()` from TanStack Router returns the current URL search params as a typed object.
- **Writing state:** `useNavigate()` is called with the updated params object whenever a filter changes — no `setState`.
- **Data fetching:** `useQuery(['products', filters])` re-fires whenever `filters` (derived from URL params) changes.
- **Debouncing:** The text search input passes through `useDebounce(value, 400)` before being written to the URL, preventing excessive requests on keystroke.
- **Pagination:** `page` and `limit` are also URL params; the `<Pagination>` component writes them via `useNavigate`.



Cart System — Dual-Layer Architecture

The cart operates with two layers that stay in sync: a Zustand store (for instant UI access, e.g. header badge) and a TanStack Query cache (authoritative server data).



- **CartDrawer** is a slide-over panel rendered at the root layout level, toggled by `cartStore.isOpen`.
- **Guest sessions:** The `x-session-id` UUID in `localStorage` ties anonymous cart data on the server to this browser tab.
- **Mutation side-effects:** All add/remove/update cart mutations call `queryClient.invalidateQueries(['cart'])` on success to trigger a refetch and re-sync the Zustand store.



CheckoutForm — Dependency Map

HOOK / DEPENDENCY	PURPOSE
<code>useQuery(['cart'])</code>	Loads CartSummary to populate OrderSummary sidebar (items + totals)
<code>useMutation(createOrder)</code>	POSTs address → receives <code>clientSecret</code> , <code>orderId</code> , <code>amount</code>
<code>useMutation(confirmOrder)</code>	POSTs payment confirmation; <code>onSuccess</code> clears cart and navigates
<code>useCartStore().clearLocalCart()</code>	Clears Zustand cart store after order success (Bug #10 fix)
<code>queryClient.invalidateQueries</code>	Forces cart refetch after checkout to sync server state (Bug #10 fix)
<code>loadStripe()</code> + <code><Elements></code>	Wraps PaymentForm with Stripe.js context when clientSecret is real
<code>clientSecret.startsWith('mock_')</code>	Demo Mode guard — bypasses Stripe Elements (Bug #9 fix)

 Auth Store — Consumer Map

The `useAuth()` hook is a thin selector wrapper over `useAuthStore()` that exposes computed helpers alongside raw state.

CONSUMER	USES FROM AUTH STORE
<code>Header</code>	<code>user</code> , <code>isAuthenticated</code> , <code>clearAuth</code> , <code>isAdmin()</code> , <code>isVendor()</code>
<code>ProtectedRoute</code>	<code>isAuthenticated</code> — redirects to login if false
<code>RoleGuard</code>	<code>user.role</code> — redirects if role does not match required
<code>VendorGate</code>	<code>user.role</code> — shows DashboardLayout or apply page
<code>api.ts</code> (interceptor)	<code>accessToken</code> — injected into every outgoing request

CONSUMER	USES FROM AUTH STORE
VendorDashboard	updateUser() — refreshes role after vendor application (Bug #18 fix)

Enterprise Multi-Vendor E-Commerce Marketplace — Frontend Reference

Stack: React 19 · TypeScript · Vite · TanStack Router · TanStack Query · Zustand · Axios · Stripe.js · Tailwind CSS

Generated June 11, 2026 · Internal documentation · Not for distribution